Design and Implementation of a Data Warehouse for Quality and Sustainability Analysis

Course: Data Management and Business Intelligence

Student: Farouk Fadel Brachemi

Company: Loacker

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Domain Analysis and Description **2**
2.1 Overview of the Company . . . 2
2.2 Why Data Warehouse? . . . 3
2.3 Business Process Description . . . 3

3 Conceptual Design **4**
3.1 Bus Matrix . . . 4
3.2 Dimensional Fact Model (DFM) . . . 4
3.3 Fact Description . . . 5
3.4 Dimensions Description . . . 6

4 Logical Design **7**
4.1 Star Schema . . . 7
4.2 Example Table Instances . . . 8
4.2.1 `date_dim` . . . 8
4.2.2 `product_dim` . . . 8
4.2.3 `plant_dim` . . . 8
4.2.4 `batch_dim` . . . 9
4.2.5 `fact_batch_quality_sustainability` . . . 9
4.3 Business Questions and SQL Queries . . . 9
4.3.1 Business Question 1 . . . 10
4.3.2 Business Question 2 . . . 11

5 Physical Design & ETL **11**
5.1 Physical Design in PostgreSQL . . . 12
5.2 ETL Approach . . . 12
5.3 Extract . . . 12
5.4 Transform and Clean . . . 13
5.5 Load . . . 13

6 Querying **13**
6.1 OLAP Queries . . . 14
6.1.1 ROLLUP . . . 14
6.1.2 CUBE . . . 15
6.1.3 GROUPING SETS . . . 16
6.2 Advanced Analytical Queries . . . 17
6.2.1 Ranking Query . . . 17
6.2.2 Windowing Query . . . 18
6.2.3 Period-to-Period Comparison . . . 19

7 Data Visualization **21**

8 Conclusion **22**

1 Introduction

In modern organizations, data has become a strategic asset that supports informed decision-making at both operational and managerial levels. As companies grow and operate across multiple locations, the volume, variety, and complexity of their data increase significantly. In such contexts, traditional transactional systems are not sufficient to support advanced analytical requirements, making data warehouses a fundamental component of modern information systems.

This project focuses on the design and implementation of a data warehouse for analyzing quality and sustainability data in a manufacturing context inspired by a global food company. The goal is to create a single system that allows the analysis of production quality, environmental impact, and performance across different plants, products, and time periods.

The project follows the main steps of a data warehouse lifecycle, including domain analysis, data modeling, physical implementation, and ETL processes. A star schema is used to organize the data, and an ETL process is applied to clean and transform raw data into a format ready for analysis. The final data warehouse supports analytical queries and helps evaluate quality and sustainability indicators.

2 Domain Analysis and Description

2.1 Overview of the Company

Loacker is an international food manufacturing company known for producing high-quality confectionery products such as wafers, chocolate, and snack items. The company was founded in Italy and operates several production plants in different countries, supplying products to markets around the world. Loacker focuses strongly on product quality, innovation, and sustainability.

As a manufacturing company, Loacker generates large amounts of data every day. This data comes from production activities, quality control systems, and sustainability monitoring. Examples of such data include production volumes, quality test results, energy and water usage, waste generation, and carbon emissions. This data is usually stored in different operational systems and formats, which makes it difficult to analyze in a unified way.

2.2 Why Data Warehouse?

The main motivation for developing a data warehouse for Loacker is the need for integrated and reliable analysis across the company. Operational systems are designed to support daily activities, but they are not suitable for complex analytical queries or historical analysis.

A data warehouse allows Loacker to collect data from different sources and store it in a single, structured repository optimized for analysis. This makes it possible to analyze quality and sustainability indicators across plants, products, and time periods. It also supports trend analysis, comparisons between locations, and long-term monitoring of environmental impact. Such insights are important for management decisions, regulatory compliance, and sustainability reporting.

2.3 Business Process Description

The business process selected is the **Quality and Sustainability Analysis** of production activities. This process focuses on monitoring the quality of products and the environmental impact of production at the batch level.

Each production batch represents a specific production event where a product is produced in a plant on a given day and shift. For each batch, quality and sustainability data is recorded. This includes the number of units produced, the number of rejected units, quality test scores, and environmental indicators such as carbon emissions, waste, energy consumption, and water usage. The main business questions addressed by this process are:

- **How does product quality vary across different production plants?**
- **Which product lines have the highest carbon emissions and waste?**
- **How do quality and sustainability indicators change over time?**

The Key Performance Indicators (KPIs) needed for this questions are:

- **Units produced**
- **Units rejected**
- **Quality test score**
- **Carbon emissions (kg CO_2e)**
- **Waste generated (kg)**
- **Energy consumption (kWh)**
- **Water usage (liters)**

The chosen level of granularity is one row per production batch, per product, per plant, per day. This granularity is detailed enough to support operational analysis while also allowing aggregation for higher-level management and sustainability reporting.

3 Conceptual Design

The conceptual design of the data warehouse follows the Dimensional Fact Model (DFM), which is well suited for analytical processing and decision support. The model is centered around a single fact table that represents the main business process, and several dimensions that provide descriptive and contextual information for analysis.

The goal of the conceptual design is to clearly identify the fact, its measures, the related dimensions, and the relationships between them. This design supports aggregation, drill-down, and comparison of quality and sustainability indicators across different products, plants, batches, and time periods.

3.1 Bus Matrix

The bus matrix defines the relationship between business processes and dimensions and helps identify conformed dimensions shared across processes. In this project, two business processes are considered: **Sales** and **Quality & Sustainability.**

Business Process	**Date**	**Product**	**Store**	**Geography**	**Batch**
Sales	Yes	Yes	Yes	Yes	No
Quality & Sustainability	Yes	Yes	No	Yes	Yes

Table 1: Bus matrix showing dimensions used by each business process

The Date, Product, and Geography(Plant) dimensions are conformed dimensions shared by both processes, which enables consistent analysis across sales and production data.

3.2 Dimensional Fact Model (DFM)

Figure 1 shows the Dimensional Fact Model of the Quality and Sustainability process. The model is organized as a star schema with the Batch Quality and Sustainability fact at the center and the Date, Product, Plant, and Batch dimensions connected to it.

Each dimension includes hierarchical levels that support aggregation and drill-down operations. Multiple hierarchies are defined where appropriate, such as in the Date and Product dimensions, to allow flexible analysis. The chosen granularity of the model ensures a balance between detailed analysis and efficient aggregation.

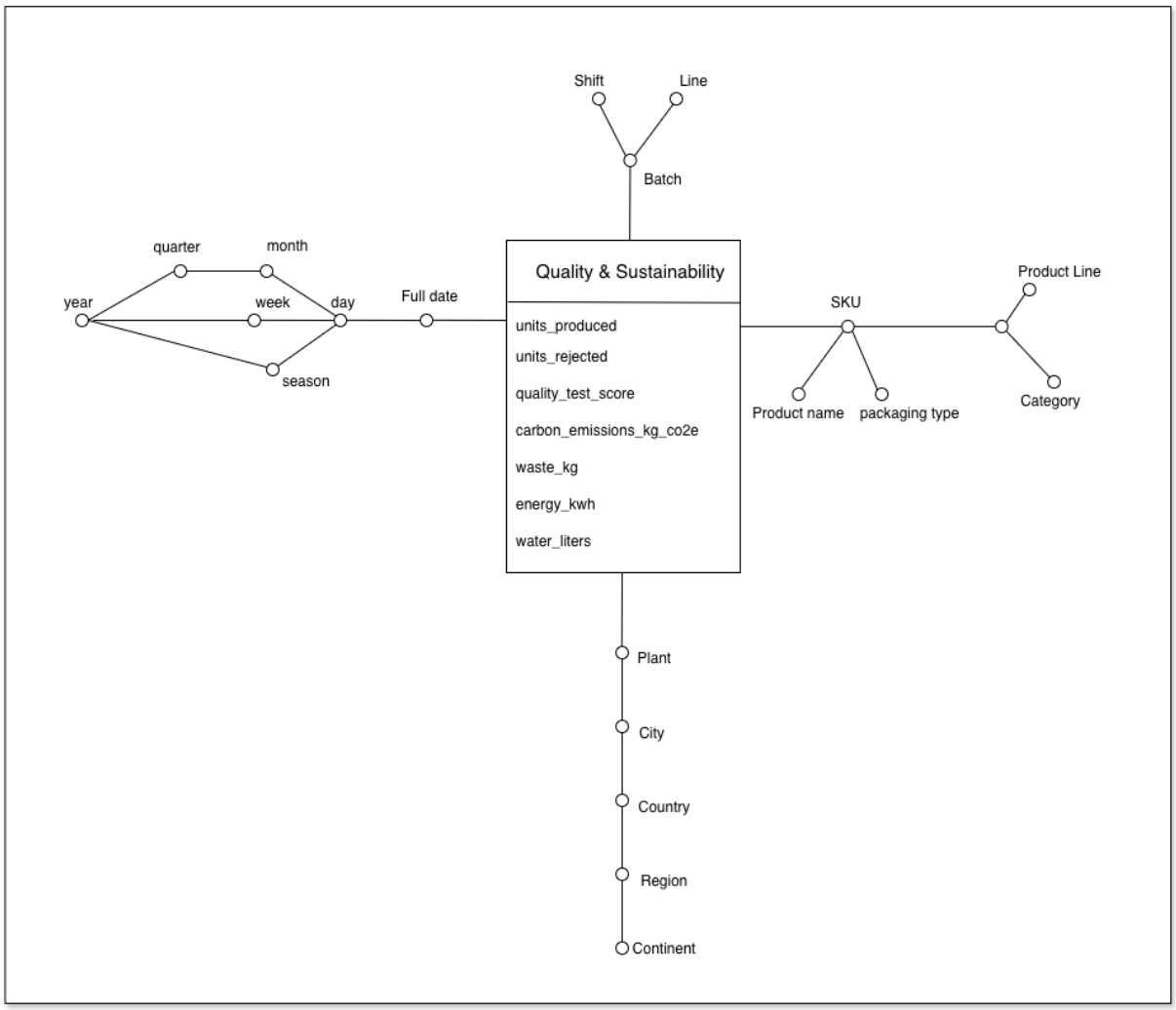

Figure 1: Dimensional Fact Model for the Quality and Sustainability business process

3.3 Fact Description

The central fact of the data warehouse is the **Batch Quality and Sustainability** fact. Each fact record represents one production batch of a specific product, produced in a specific plant on a specific day. The fact table contains the following measures:

- **Units Produced**: total number of units produced in a batch
- **Units Rejected**: number of units rejected due to quality issues
- **Quality Test Score**: numerical score representing product quality
- **Carbon Emissions (kg CO_2e)**: emissions generated during production
- **Waste (kg)**: amount of waste generated
- **Energy Consumption (kWh)**: energy used during production
- **Water Usage (liters)**: water consumed during production

Most measures in the fact table are additive, meaning they can be summed across all dimensions such as time, product, plant, and batch. These include units produced, units rejected, carbon emissions, waste, energy consumption, and water usage.

The quality test score is a non-additive measure. It cannot be summed meaningfully across dimensions and is typically aggregated using average or minimum/maximum values. This distinction is important to ensure correct analytical results.

3.4 Dimensions Description

The fact is analyzed through four dimensions:

- **Date Dimension**: provides temporal context and supports analysis by day, month, quarter, year, week, and season. It enables trend analysis and seasonality studies.
- **Product Dimension**: describes the products being produced. The lowest level is the SKU, with roll-up hierarchies to product line and category. Packaging type and product name are descriptive attributes.
- **Plant Dimension**: represents the production locations. It includes a geographical hierarchy from plant to city, country, region, and continent, enabling global comparison.
- **Batch Dimension**: represents individual production batches. It includes operational attributes such as batch number, shift, and production line, supporting detailed operational analysis.

4 Logical Design

In this task, the conceptual DFM is translated into a logical model using a star schema. The star schema contains one central fact table and four dimension tables. The fact table stores the measures for the Quality & Sustainability process, while the dimensions provide context for analysis (time, product, plant, and batch).

4.1 Star Schema

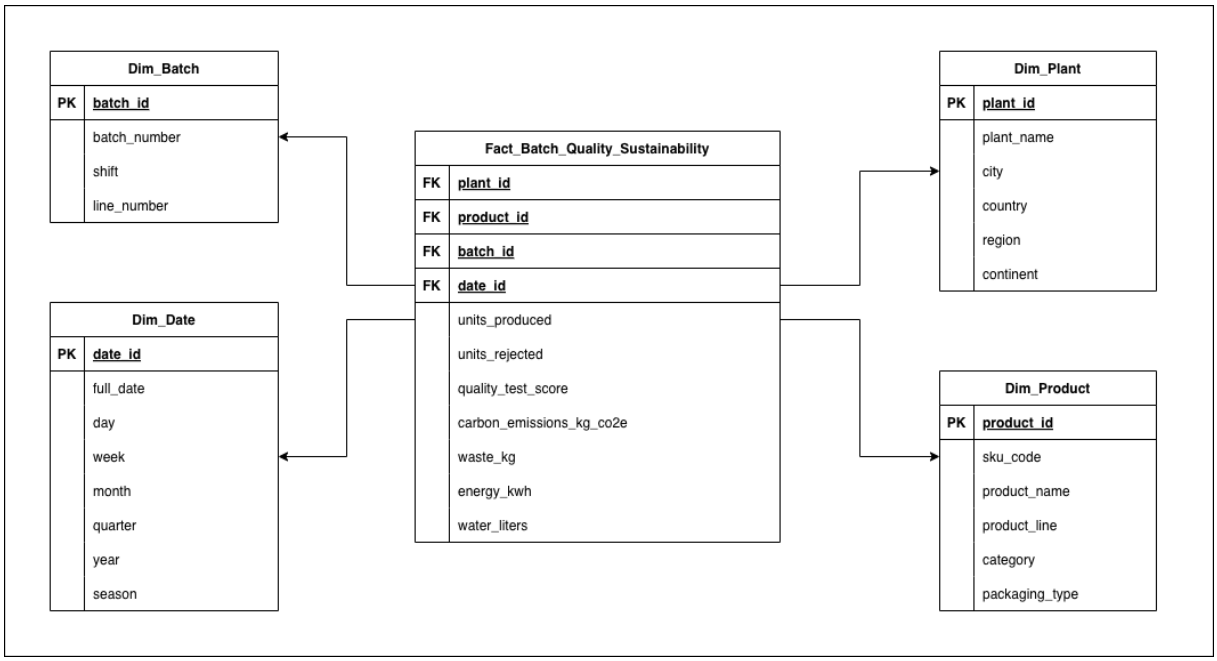

Figure 2: Star schema for the Quality and Sustainability data warehouse

Figure 2 shows the logical design of the data warehouse implemented as a star schema. The schema is centered around the `fact_batch_quality_sustainability` table, which stores the measures related to production quality and sustainability at the batch level.

The fact table contains foreign keys that reference four dimension tables: Date, Product, Plant, and Batch. These dimensions provide the contextual information required to analyze the measures across time, products, production locations, and operational conditions. The grain of the fact table is one row per production batch, per product, per plant, and per day, ensuring a consistent and well-defined level of detail.

The Date dimension supports time-based analysis through multiple hierarchies, including daily, monthly, yearly, weekly, and seasonal aggregations. The Product dimension allows analysis at the SKU level, with roll-ups to product line and category. The Plant dimension provides a geographical hierarchy from plant to continent, enabling global comparison. The Batch dimension captures operational details such as batch number, shift, and production line, supporting detailed operational analysis.

This star schema structure is optimized for analytical queries and aggregation. It ensures simplicity, clarity, and efficient query performance, making it suitable for answering the business questions related to quality and sustainability analysis.

4.2 Example Table Instances

To demonstrate the logical model, small example instances of each table are presented. The examples include 3 dates, 3 products, 2 plants, 5 batches, and 12 fact rows.

4.2.1 `date_dim`

	date_id [PK] integer	full_date date	day integer	month integer	year integer	season character varying (20)
1	20230101	2023-01-01	1	1	2023	Winter
2	20230102	2023-01-02	2	1	2023	Winter
3	20230103	2023-01-03	3	1	2023	Winter

Figure 3: Example rows from `date_dim`

4.2.2 `product_dim`

	product_id [PK] bigint	sku_code character var	product_name character varying (255)	product_line character varyin	category character var	packaging_type character varying
1	1	SKU006	Loacker Dark Choco	Chocolate	Sweets	Wrapper
2	2	SKU007	Loacker Protein Bar	Bars	Health	Wrapper
3	3	SKU008	Loacker Family Pack	Assortments	Snacks	Box

Figure 4: Example rows from `product_dim`

4.2.3 `plant_dim`

	plant_id [PK] bigint	plant_name character varying	city character var	country character var	region character var	continent character var
1	1	Plant Bolzano	Bolzano	Italy	SÜDtirol	Europe
2	2	Plant Munich	Munich	Germany	Bavaria	Europe

Figure 5: Example rows from `plant_dim`

4.2.4 `batch_dim`

	batch_id [PK] bigint	batch_number character varying (80)	shift character var	line_number character varyir
1	1	B-20230403-0005938	B	L1
2	2	B-20230418-0006904	B	L2
3	3	B-20230807-0014016	B	L2
4	4	B-20250419-0053728	B	L1
5	5	B-20240601-0033152	B	L1

Figure 6: Example rows from `batch_dim`

4.2.5 `fact_batch_quality_sustainability`

	full_date date	plant_name character varying	sku_code character var	batch_number character varying (80)	units_produced integer	units_rejected integer	quality_test_score numeric (10,4)	carbon_emissions_kg_co2e numeric (18,6)
1	2023-01-01	Plant Bolzano	SKU001	B-20230101-0000002	1269	20	92.2200	49.169000
2	2023-01-01	Plant Bolzano	SKU001	B-20230101-0000001	2109	94	75.1700	66.535000
3	2023-01-01	Plant Bolzano	SKU002	B-20230101-0000004	1927	83	74.8500	140.440000
4	2023-01-01	Plant Bolzano	SKU002	B-20230101-0000003	1290	72	73.8900	90.163000
5	2023-01-01	Plant Bolzano	SKU003	B-20230101-0000005	0	0	83.0200	62.891000
6	2023-01-01	Plant Bolzano	SKU003	B-20230101-0000006	2171	31	93.2400	67.472000
7	2023-01-01	Plant Bolzano	SKU004	B-20230101-0000007	1230	37	82.8200	62.083000
8	2023-01-01	Plant Bolzano	SKU004	B-20230101-0000008	1792	35	87.5400	73.392000
9	2023-01-01	Plant Bolzano	SKU005	B-20230101-0000010	1299	36	90.5100	65.837000
10	2023-01-01	Plant Bolzano	SKU005	B-20230101-0000009	1516	8	77.8800	91.621000
11	2023-01-01	Plant Bolzano	SKU006	B-20230101-0000012	1624	32	95.1200	75.789000
12	2023-01-01	Plant Bolzano	SKU006	B-20230101-0000011	1761	43	83.9900	46.444000

Figure 7: Example rows from the fact table

4.3 Business Questions and SQL Queries

To validate the logical design of the data warehouse and demonstrate its analytical capabilities, a set of SQL queries was executed on the implemented star schema using PostgreSQL. These queries are based on key business questions related to production quality and sustainability and reflect typical analytical scenarios faced by decision makers.

For each business question, the corresponding SQL query and its result are presented. These results illustrate how the data warehouse supports efficient analysis and provides clear insights into quality and sustainability performance.

4.3.1 Business Question 1

Which plant has better production quality in December 2025, considering both average quality test score and reject rate?

SQL query:

```
SELECT
  pl.country,
  pl.plant_name,
  d.year,
  d.month,
  ROUND(AVG(f.quality_test_score), 2) AS avg_quality_score,
  SUM(f.units_produced) AS total_units_produced,
  SUM(f.units_rejected) AS total_units_rejected,
  ROUND(
    SUM(f.units_rejected)::numeric / NULLIF(SUM(f.units_produced), 0),
    4
  ) AS reject_rate
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN date_dim  d  ON d.date_id  = f.date_id
WHERE d.year = 2025 and d.month = 12
GROUP BY pl.country, pl.plant_name, d.year, d.month
ORDER BY d.year, d.month, reject_rate ASC, avg_quality_score DESC;
```

Result:

	country character var	plant_name character varying	year integer	month integer	avg_quality_score numeric	total_units_produced bigint	total_units_rejected bigint	reject_rate numeric
1	Austria	Plant Vienna	2025	12	86.84	740961	15757	0.0213
2	Italy	Plant Bolzano	2025	12	87.76	745410	15954	0.0214
3	Germany	Plant Munich	2025	12	87.68	717578	15405	0.0215
4	USA	Plant Chicago	2025	12	86.94	737765	16297	0.0221

Figure 8: Quality performance of production plants in December 2025

The results indicate that no single plant outperforms the others across both quality dimensions. Plant Vienna shows the lowest reject rate, indicating the most efficient production in terms of waste reduction. In contrast, Plant Bolzano achieves the highest average quality test score, suggesting superior product quality. Plant Munich performs consistently well but does not lead in either metric. This highlights a trade-off between quality scores and reject rates across plants in December 2025.

4.3.2 Business Question 2

Which product line has the highest carbon footprint per unit produced?

SQL query:

```
SELECT
  p.product_line,
  SUM(f.carbon_emissions_kg_co2e) AS total_emissions_kg_co2e,
  SUM(f.units_produced) AS total_units_produced,
  ROUND(
    SUM(f.carbon_emissions_kg_co2e) / NULLIF(SUM(f.units_produced), 0),
    6
  ) AS co2_per_unit
FROM fact_batch_quality_sustainability f
JOIN product_dim p ON p.product_id = f.product_id
WHERE f.units_produced > 0
GROUP BY p.product_line
ORDER BY co2_per_unit DESC;
```

Result:

	product_line character varying (100)	total_emissions_kg_co2e numeric	total_units_produced bigint	co2_per_unit numeric
1	Assortments	1097723.066000	25924327	0.042343
2	Wafers	1651872.186000	39056151	0.042295
3	Bars	545406.267000	12924956	0.042198
4	Chocolate	1103584.427000	26159271	0.042187

Figure 9: Carbon emissions per unit produced by product line

The results show that the Assortments product line has the highest carbon emissions per unit produced, indicating a slightly higher environmental impact compared to the other product lines. In contrast, Chocolate exhibits the lowest emissions per unit, suggesting more efficient or less resource-intensive production processes. Overall, the differences between product lines are relatively small, reflecting a generally consistent sustainability performance across the product portfolio.

5 Physical Design & ETL

This task describes how the data warehouse was implemented in PostgreSQL and how the data was loaded from a CSV file into the star schema. The physical design follows the logical star schema and includes staging tables to support data cleaning and transformation before loading the final dimension and fact tables.

5.1 Physical Design in PostgreSQL

The database was implemented using five main tables: four dimension tables (`date_dim`, `product_dim`, `plant_dim`, `batch_dim`) and one fact table (`fact_batch_quality_sustainability`). Surrogate keys were used for the dimensions (`product_id`, `plant_id`, `batch_id`), while the date dimension uses a numeric key `date_id` in the format `YYYYMMDD`.

The fact table stores one record per production batch, per product, per plant, and per day (granularity at batch level). Foreign keys enforce referential integrity between the fact and dimension tables. A uniqueness constraint on the fact table ensures that the grain is not violated by duplicate records.

To improve query performance, indexes were added on the foreign keys of the fact table (`date_id`, `product_id`, `plant_id`, `batch_id`). This supports faster joins between the fact and dimensions, which is important for analytical queries.

5.2 ETL Approach

Data was generated using a Python script and exported as a CSV file, then processed using SQL-based ETL steps in PostgreSQL. As shown in Figure 10, the ETL process transforms the raw CSV data through multiple layers into a cleaned and structured star schema.

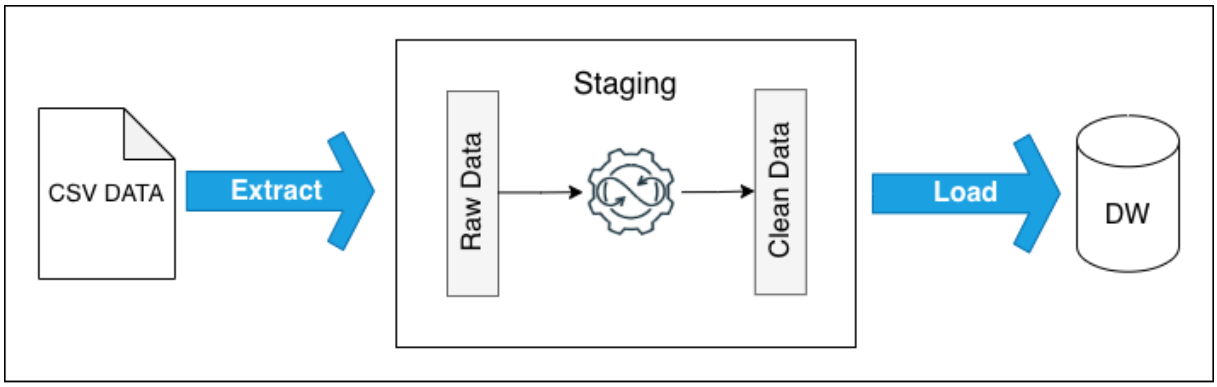

Figure 10: ETL process overview

- **Staging (Raw):** `stg.batch_data_raw` stores the CSV data exactly as received, with all attributes stored as text. This avoids loading errors and preserves the original content.
- **Staging (Clean):** `stg.batch_data_clean` contains cleaned and typed data. In this layer, formats are standardized, invalid values are corrected, and duplicates are removed.
- **Data Warehouse (Star Schema):** dimension and fact tables are populated from the clean staging table.

5.3 Extract

The extraction step consists of loading the CSV file into the raw staging table using the PostgreSQL `COPY` command. Since all columns in the raw staging table are stored as `TEXT`, the import is robust and does not fail due to datatype issues.

5.4 Transform and Clean

The transformation step converts the raw text fields into standardized and consistent values. The main cleaning rules are:

- **Text normalization:** all text fields are trimmed to remove leading/trailing spaces, and consistent casing is applied (e.g., upper-case for codes such as SKU and shift, and title case for names such as plant and city).
- **Standardization mapping:** country, region, and plant names are mapped to a consistent set of values (e.g., `Italia` → `Italy`, `Deutschland` → `Germany`).
- **Safe numeric parsing:** numeric fields (e.g., emissions, waste, energy, water) are converted from text to numeric types. If a value has an invalid format, it is replaced with a default value to avoid ETL failures.
- **Outlier correction:** negative values for measures are corrected to zero. The quality test score is constrained to the range 0–100. The number of rejected units is limited so it cannot exceed the number of produced units.
- **Deduplication:** duplicate rows are removed to ensure that each production batch is represented only once at the defined grain. Deduplication is performed using a window function and keeping only the first row per grain key (date, batch number, plant, SKU, shift, and line).

These rules produce a clean dataset that can be safely loaded into the star schema and ensures that the final warehouse tables contain consistent values.

5.5 Load

After cleaning, the data is loaded into the star schema in the following order:

1. **Load `date_dim`:** generated from distinct production dates in the clean staging table. The season attribute is derived from the month (Winter, Spring, Summer, Autumn).
2. **Load `product_dim`, `plant_dim`, `batch_dim`:** loaded using `SELECT DISTINCT` from the clean staging table.
3. **Load the fact table:** each clean record is joined to the four dimensions and inserted into `fact_batch_quality_sustainability`. A uniqueness constraint prevents inserting duplicate fact rows.

During development and repeated runs, the tables were truncated and reloaded. The fact table must be truncated before the dimension tables due to foreign key constraints.

6 Querying

This task demonstrates advanced analytical querying capabilities supported by the data warehouse. SQL queries are presented in textual form, while the corresponding results are shown using screenshots captured from PostgreSQL.

6.1 OLAP Queries

6.1.1 ROLLUP

Business Question: How do total units produced aggregate by plant and year, including subtotals and the overall total?

SQL Query:

```
SELECT
  pl.plant_name,
  d.year,
  SUM(f.units_produced) AS total_units_produced
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN date_dim d   ON d.date_id  = f.date_id
GROUP BY ROLLUP (pl.plant_name, d.year)
ORDER BY pl.plant_name, d.year;
```

Result:

	plant_name character varying (255)	year integer	total_units_produced bigint
1	Plant Bolzano	2023	8639827
2	Plant Bolzano	2024	8686134
3	Plant Bolzano	2025	8658477
4	Plant Bolzano	[null]	25984438
5	Plant Chicago	2023	8689813
6	Plant Chicago	2024	8669084
7	Plant Chicago	2025	8683429
8	Plant Chicago	[null]	26042326
9	Plant Munich	2023	8724381
10	Plant Munich	2024	8633799
11	Plant Munich	2025	8642625
12	Plant Munich	[null]	26000805
13	Plant Vienna	2023	8671131
14	Plant Vienna	2024	8710498
15	Plant Vienna	2025	8655507
16	Plant Vienna	[null]	26037136
17	[null]	[null]	104064705

Figure 11: Aggregation of total units produced by plant and year using ROLLUP

6.1.2 CUBE

Business Question: How do carbon emissions vary across combinations of plants and product lines?

SQL Query:

```
SELECT
  pl.plant_name,
  p.product_line,
  SUM(f.carbon_emissions_kg_co2e) AS total_emissions
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN product_dim p ON p.product_id = f.product_id
GROUP BY CUBE (pl.plant_name, p.product_line)
ORDER BY pl.plant_name, p.product_line;
```

Result:

	plant_name character varying	product_line character varying	total_emissions numeric
1	Plant Bolzano	Assortments	272333.278000
2	Plant Bolzano	Bars	131980.303000
3	Plant Bolzano	Chocolate	268881.740000
4	Plant Bolzano	Wafers	408826.369000
5	Plant Bolzano	[null]	1082021.690000
6	Plant Chicago	Assortments	311675.234000
7	Plant Chicago	Bars	156652.577000
8	Plant Chicago	Chocolate	314369.149000
9	Plant Chicago	Wafers	470878.836000
10	Plant Chicago	[null]	1253575.796000
11	Plant Munich	Assortments	241002.506000
12	Plant Munich	Bars	121308.183000
13	Plant Munich	Chocolate	245104.673000
14	Plant Munich	Wafers	363076.365000
15	Plant Munich	[null]	970491.727000

16	Plant Vienna	Assortments	283983.656000
17	Plant Vienna	Bars	141533.608000
18	Plant Vienna	Chocolate	286368.001000
19	Plant Vienna	Wafers	426842.654000
20	Plant Vienna	[null]	1138727.919000
21	[null]	Assortments	1108994.674000
22	[null]	Bars	551474.671000
23	[null]	Chocolate	1114723.563000
24	[null]	Wafers	1669624.224000
25	[null]	[null]	4444817.132000

Figure 12: Carbon emissions aggregated by plant and product line using CUBE

6.1.3 GROUPING SETS

Business Question: What are the total rejected units by plant and by product line, analyzed separately?

SQL Query:

```
SELECT
  pl.plant_name,
  p.product_line,
  SUM(f.units_rejected) AS total_units_rejected
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN product_dim p ON p.product_id = f.product_id
GROUP BY GROUPING SETS (
  (pl.plant_name),
  (p.product_line)
)
ORDER BY pl.plant_name, p.product_line;
```

Result:

	plant_name character varying (255)	product_line character varying (100)	total_units_rejected bigint
1	Plant Bolzano	[null]	565607
2	Plant Chicago	[null]	573039
3	Plant Munich	[null]	556352
4	Plant Vienna	[null]	567385
5	[null]	Assortments	561003
6	[null]	Bars	278553
7	[null]	Chocolate	568793
8	[null]	Wafers	854034

Figure 13: Rejected units aggregated using GROUPING SETS

6.2 Advanced Analytical Queries

6.2.1 Ranking Query

Business Question: How are production plants ranked based on their average quality test score?

SQL Query:

```
SELECT
  pl.plant_name,
  ROUND(AVG(f.quality_test_score), 2) AS avg_quality_score,
  RANK() OVER (ORDER BY AVG(f.quality_test_score) DESC) AS quality_rank
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
GROUP BY pl.plant_name
ORDER BY quality_rank;
```

Result:

	plant_name character varying (255)	avg_quality_score numeric	quality_rank bigint
1	Plant Munich	86.60	1
2	Plant Bolzano	86.03	2
3	Plant Vienna	85.81	3
4	Plant Chicago	85.59	4

Figure 14: Ranking of plants by average quality test score

6.2.2 Windowing Query

Business Question: How does each plant's monthly production compare to its overall average monthly production?

SQL Query:

```
SELECT
  pl.plant_name,
  d.year,
  d.month,
  SUM(f.units_produced) AS monthly_units,
  AVG(SUM(f.units_produced)) OVER (
    PARTITION BY pl.plant_name
  ) AS avg_monthly_units
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN date_dim d   ON d.date_id  = f.date_id
GROUP BY pl.plant_name, d.year, d.month
ORDER BY pl.plant_name, d.year, d.month;
```

Result:

	plant_name character varying (255)	year integer	month integer	monthly_units bigint	avg_monthly_units numeric
1	Plant Bolzano	2023	1	739971	721789.944444444444
2	Plant Bolzano	2023	2	658758	721789.944444444444
3	Plant Bolzano	2023	3	741359	721789.944444444444
4	Plant Bolzano	2023	4	698331	721789.944444444444
5	Plant Bolzano	2023	5	734941	721789.944444444444
6	Plant Bolzano	2023	6	715642	721789.944444444444
7	Plant Bolzano	2023	7	742411	721789.944444444444
8	Plant Bolzano	2023	8	736255	721789.944444444444
9	Plant Bolzano	2023	9	699069	721789.944444444444
10	Plant Bolzano	2023	10	731487	721789.944444444444
11	Plant Bolzano	2023	11	703120	721789.944444444444
12	Plant Bolzano	2023	12	738483	721789.944444444444
13	Plant Bolzano	2024	1	732642	721789.944444444444

	plant_name character varying (255)	year integer	month integer	monthly_units bigint	avg_monthly_units numeric
132	Plant Vienna	2024	12	752484	723253.777777777778
133	Plant Vienna	2025	1	734514	723253.777777777778
134	Plant Vienna	2025	2	663258	723253.777777777778
135	Plant Vienna	2025	3	732833	723253.777777777778
136	Plant Vienna	2025	4	707095	723253.777777777778
137	Plant Vienna	2025	5	714401	723253.777777777778
138	Plant Vienna	2025	6	722080	723253.777777777778
139	Plant Vienna	2025	7	726685	723253.777777777778
140	Plant Vienna	2025	8	729054	723253.777777777778
141	Plant Vienna	2025	9	726117	723253.777777777778
142	Plant Vienna	2025	10	741190	723253.777777777778
143	Plant Vienna	2025	11	717319	723253.777777777778
144	Plant Vienna	2025	12	740961	723253.777777777778

Figure 15: Comparison of monthly production with plant-level averages

6.2.3 Period-to-Period Comparison

Business Question: How does monthly production compare with the same month in the previous year?

SQL Query:

```
SELECT
  pl.plant_name,
  d.year,
  d.month,
  SUM(f.units_produced) AS current_year_units,
  LAG(SUM(f.units_produced)) OVER (
    PARTITION BY pl.plant_name, d.month
    ORDER BY d.year
  ) AS previous_year_units,
  SUM(f.units_produced)
  - LAG(SUM(f.units_produced)) OVER (
      PARTITION BY pl.plant_name, d.month
      ORDER BY d.year
    ) AS year_over_year_change
FROM fact_batch_quality_sustainability f
JOIN plant_dim pl ON pl.plant_id = f.plant_id
JOIN date_dim d   ON d.date_id  = f.date_id
GROUP BY pl.plant_name, d.year, d.month
ORDER BY pl.plant_name, d.month, d.year;
```

Result:

	plant_name character varying (255)	year integer	month integer	current_year_units bigint	previous_year_units bigint	year_over_year_change bigint
1	Plant Bolzano	2023	1	739971	[null]	[null]
2	Plant Bolzano	2024	1	732642	739971	-7329
3	Plant Bolzano	2025	1	745411	732642	12769
4	Plant Bolzano	2023	2	658758	[null]	[null]
5	Plant Bolzano	2024	2	681391	658758	22633
6	Plant Bolzano	2025	2	662304	681391	-19087
7	Plant Bolzano	2023	3	741359	[null]	[null]
8	Plant Bolzano	2024	3	740285	741359	-1074
9	Plant Bolzano	2025	3	737727	740285	-2558
10	Plant Bolzano	2023	4	698331	[null]	[null]
11	Plant Bolzano	2024	4	701560	698331	3229
12	Plant Bolzano	2025	4	714270	701560	12710
13	Plant Bolzano	2023	5	734941	[null]	[null]

	plant_name character varying (255)	year integer	month integer	current_year_units bigint	previous_year_units bigint	year_over_year_change bigint
132	Plant Vienna	2025	8	729054	724139	4915
133	Plant Vienna	2023	9	724894	[null]	[null]
134	Plant Vienna	2024	9	710399	724894	-14495
135	Plant Vienna	2025	9	726117	710399	15718
136	Plant Vienna	2023	10	751833	[null]	[null]
137	Plant Vienna	2024	10	746820	751833	-5013
138	Plant Vienna	2025	10	741190	746820	-5630
139	Plant Vienna	2023	11	702214	[null]	[null]
140	Plant Vienna	2024	11	707724	702214	5510
141	Plant Vienna	2025	11	717319	707724	9595
142	Plant Vienna	2023	12	751000	[null]	[null]
143	Plant Vienna	2024	12	752484	751000	1484
144	Plant Vienna	2025	12	740961	752484	-11523

Figure 16: Year-over-year comparison of monthly production

7 Data Visualization

An interactive dashboard was created using Power BI to present the key outcomes of the quality and sustainability analysis. The dashboard is built on CSV data exported from the PostgreSQL data warehouse following the ETL process. It visually summarizes the main quality and sustainability indicators derived from the star schema, offering a high-level overview of production performance while enabling users to explore the data through interactive filters.

At the top of the dashboard, slicers enable filtering by year, month, product, plant, and shift. These filters allow flexible analysis across different time periods, production locations, and operational conditions. Four key performance indicators are displayed as cards: total carbon emissions, average quality score, total units produced, and total units rejected. These KPIs provide an immediate summary of quality and sustainability performance.

The dashboard includes a line chart showing monthly units produced with a year-over-year comparison. This visualization highlights production trends and allows comparison between different years. Two bar charts support sustainability analysis: one presents carbon emissions per unit by product line, and the other shows carbon emissions per unit by plant. These charts make it possible to identify product lines and plants with higher environmental impact.

Overall, the dashboard offers a clear and intuitive visualization of the most important quality and sustainability indicators. By combining KPI cards, trend analysis, and comparative charts with interactive slicers, the dashboard supports data-driven decision making and complements the analytical queries executed on the data warehouse.

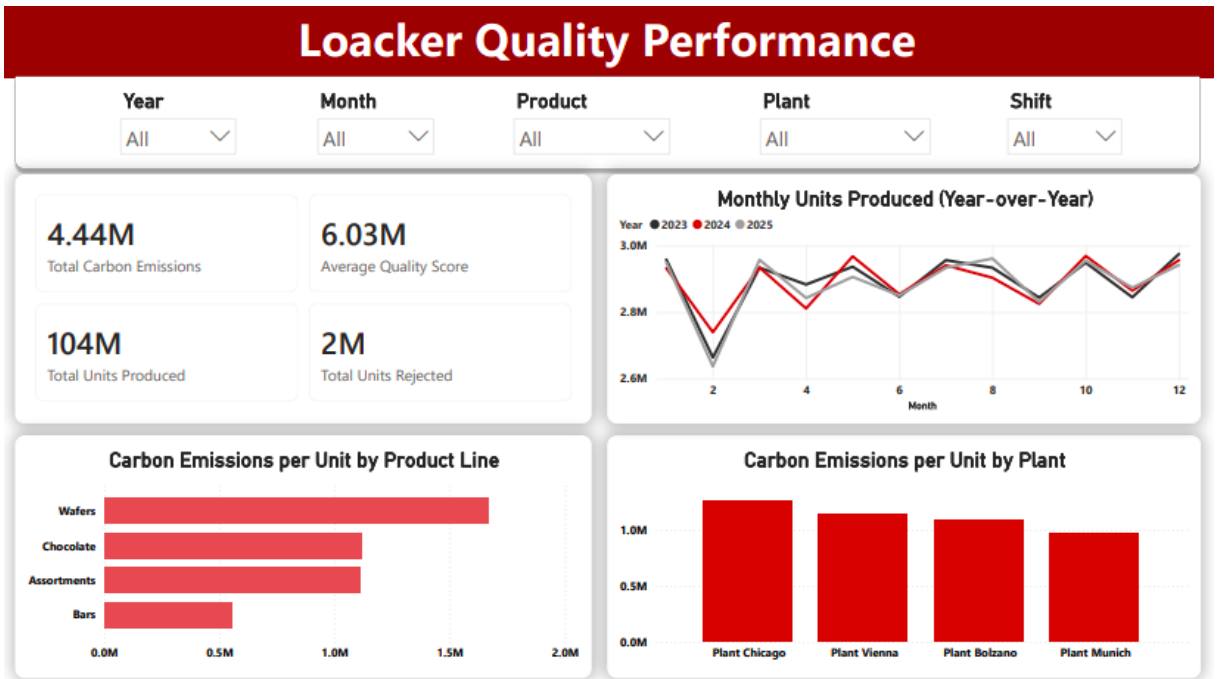

Figure 17: Power BI dashboard for quality and sustainability analysis at Loacker

8 Conclusion

This project implemented a data warehouse to support quality and sustainability analysis for Loacker's production processes. The star schema and ETL pipeline enabled reliable and efficient analytical querying, while the Power BI dashboard provided a clear and interactive visualization of key performance indicators. Overall, the project demonstrates how data warehousing and visualization can effectively support data-driven decision making in a manufacturing environment.